

УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
НОВИ САД
Департман за рачунарство и аутоматику
Одсек за рачунарску технику и рачунарске комуникације

ПРОЈЕКТНИ РАД

Кандидат: Никола Јокић
Број индекса: RA95/2024

Предмет: Основи паралелног програмирања и софтверски алати
Тема рада: МАН - преводац

Ментор рада: Милица Тадић

Нови Сад, јун, 2026.

САДРЖАЈ:

1. Анализа проблема

- 1.1 Опис проблема
- 1.2 Улаз и излаз система
- 1.3 МАНН подржане инструкције
- 1.4 Декларација у језику
- 1.5 Ограничења

2. Концепт решења

- 2.1 Архитектура преводиоца
- 2.2 Приступ решавању проблема
- 2.3 Фазе превођења

3. Програмско решење

- 3.1 Ток извршавања програма
- 3.2 Лексичка анализа
- 3.3 Синтаксна анализа
- 3.4 Граф тока управљања
- 3.5 Анализа животног века
- 3.6 Граф сметњи и алокација ресурса
- 3.7 Генерисање кода
- 3.8 Излаз програма

4. Упутство за покретање

- 4.1 Покретање на Linux-у (Ubuntu)
- 4.2 Покретање на Windows-у
- 4.3 Избор улазне датотеке

5. Коришћени тест програми

- 5.1 Тест пример 1 – simple.mavn
- 5.2 Тест пример 2 – multiply.mavn
- 5.3 Тест пример 3 – extended.mavn
- 5.4 Тест пример 4 – lex_error.mavn
- 5.5 Тест пример 5 – syn_error.mavn
- 5.6 Тест пример 6 – undefined_var.mavn
- 5.7 Тест пример 7 – unknown_instr.mavn

6. Верификација

- 6.1 Проблем
- 6.2 Доказ да је потребно 5 регистара
- 6.3 Решење
- 6.4 Провера у QTSpm симулатору

1. Анализа проблема

1.1 Опис проблема

У оквиру овог пројекта реализован је МАВН (Мипс Асемблер Високог Нивоа) преводаца који преводи програме написане на вишем асемблерском језику на основни MIPS 32bit-ни асемблерски језик.

МАВН асемблерски језик служи лакшем асемблерском програмирању јер уводи концепт регистарске променљиве које омогућавају коришћење променљивих уместо правих ресурса.

1.2 Улаз и излаз система

Улаз представља фајл са екстензијом .mavn написан на МАВН асемблерском језику, а излаз система представља фајл са екстензијом .s који садржи основни MIPS 32bit-ни асемблерски код.

Такође излаз система представљају и детектоване грешке, тачније извештај о лексичким и синтаксним грешкама.

1.3 МАВН подржане инструкције

МАВН језик подржава 10 MIPS инструкција и 3 додатно имплементирани инструкције:

1. add – сабирање две регистарске променљиве
2. addi - сабирање регистарске променљиве са константом
3. sub – одузимање две регистарске променљиве
4. la - учитавање адресе променљиве у регистар
5. li – учитавање константе у регистар
6. lw – учитавање једне меморијске речи
7. sw – упис једне меморијске речи
8. b – безусловни скок
9. bltz – скок ако је регистар мањи од нуле
10. nop – инструкција без операције
11. mul – множење две регистарске променљиве
12. and – логичко и за две регистарске променљиве
13. beq – скок ако су две регистарске променљиве једнаке

1.4 Декларација у језику

МАНН језик подржава три врсте декларација:

- **Декларација меморијске променљиве:**

_mem varName value

varName – почиње малим словом *m*, у наставку може бити било који број

Пример: *_mem m1 66*

- **Декларација регистарске променљиве:**

_reg varName

varName – почиње малим словоом *r*, у наставку може бити било који број

Пример: *_reg r1*

- **Декларација функције:**

_func funcName

funcName – мора почети словом, у наставку може бити било који низ слова и/или бројева

Пример: *_func funkcija*

1.5 Ограничења

Преводацац је ограничен на једну улазну датотеку. У инструкцијама се искључиво користе променљиве, а не прави ресурси. MIPS 32bit-ни асемблер поседује само 4 физичка регистра: *t0*, *t1*, *t2*, *t3*.

2. Концепт решења

2.1 Архитектура преводиоца

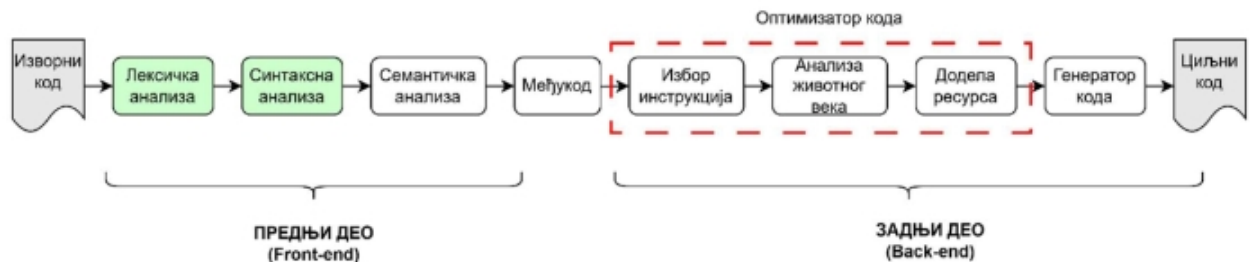
Преводацац је организован у више фаза које се извршавају секвенцијално. Свака фаза прима резултат претходне фазе и прослеђује га наредној.

1. Лексичка анализа
2. Синтаксна анализа
3. Граф тока управљања
4. Анализа животног века
5. Граф сметњи
6. Алокација ресурса
7. Генерисање кода

2.2 Приступ решавању проблема

Превођење MABH асемблерског програма на MIPS асемблер реализовано је по узору на архитектуру класичног компајлера (слика 1). Он је подељен на предњи део (front-end) и задњи део (back-end).

Предњи део се бави лексичком и синтаксном анализом док задњи део бира инструкције, врши анализу животног века, алоцира ресурсе и генерише финални код (врши оптимизацију и генерише код).



слика 1.

2.3 Фазе превођења

Проблем превођења подељен је на независне фазе које се извршавају секвенцијално, где свака фаза решава један проблем и прослеђује добијени резултат наредној фази. Овај приступ омогућава лакше тестирање, дебаговање и развој сваке компоненте.

3. Програмско решење

3.1 Ток извршавања програма

Главни фајл (main.cpp) позива све фазе одређеним редоследом:

3.2 Лексичка анализа

Лексичка анализа је реализована помоћу коначног детерминистичког аутомата (FSM) са 45 стања. Тај аутомат је формално дефинисан као коначан скуп: $D = (\Sigma, Q, q_0, F, \delta)$ где је:

- Σ – коначна азбука
- Q – коначан скуп стања
- q_0 – почетно стање
- F – скуп прихватљивих (финалних) стања
- δ – функција прелаза која одређује транзиције између стања

Аутомат чита улазну датотеку, препознаје токене и враћа листу токена.

3.3 Синтаксна анализа

Синтаксна анализа је део компајлера који проверава да ли су низови симбола добијени из лексичке анализе у складу са граматиком језика (слика 2).

$Q \rightarrow S ; L$	$S \rightarrow _mem \ mid \ num$	$L \rightarrow eof$	$E \rightarrow add \ rid, \ rid, \ rid$
$S \rightarrow _reg \ rid$	$L \rightarrow Q$		$E \rightarrow addi \ rid, \ rid, \ num$
$S \rightarrow _func \ id$			$E \rightarrow sub \ rid, \ rid, \ rid$
$S \rightarrow id : E$			$E \rightarrow la \ rid, \ mid$
$S \rightarrow E$			$E \rightarrow lw \ rid, \ num(rid)$
			$E \rightarrow li \ rid, \ num$
			$E \rightarrow sw \ rid, \ num(rid)$
			$E \rightarrow b \ id$
			$E \rightarrow bltz \ rid, \ id$
			$E \rightarrow nop$

слика 2.

3.4 Граф тока управљања

Први корак у анализи животног века је конструисање графа тока управљања који садржи чворове (инструкције) и гране (ток управљања програмом). Граф тока управљања се конструира тако што се за сваку инструкцију дефинише чвор, а за свака два чвора x и y (x претходи y) повлачи се стрелица од x ка y .

3.5 Анализа животног века

Анализа животног века је анализа тока података која за сваку тачку програма одређује да ли је вредност неке променљиве у тој тачки потребна у наставку извршавања. За сваку инструкцију дефинишу се следећи скупови:

- Скуп претходника $pred[n]$
- Скуп наследника $succ[n]$
- Скуп променљивих које чвор користи $use[n]$
- Скуп променљивих које чвор дефинише $def[n]$

Резултат анализе су следећи скупови:

- $in[n]$ – променљиве које морају бити доступне пре извршавања чвора
- $out[n]$ – променљиве које морају бити сачуване након извршавања чвора

3.6 Граф сметњи и алокација ресурса

Граф сметњи се гради на основу анализе животног века. Две променљиве које су истовремено „живе“ добијају грану између себе у графу, што значи да не могу делити исти регистар.

Алокација ресурса користи алгоритам бојења графа са 4 боје које представљају 4 регистра доступна у MIPS 32bit-ној архитектури (t0, t1, t2, t3). Свака променљива добија регистар који се разликује од регистра свих променљивих са којима је та променљива истовремено „жива“, то ј у конфликту.

У случају да је више од 4 регистра потребно у једном тренутку бојење са 4 боје није могуће без додатне мере. Зато је у проводилац уграђен механизам spilling-a.

Механизам ради по принципу да ако променљива која не може добити ниједну од расположивих боја бива означена као просута (spilled) и њена вредност се чува у меморији, уместо да алокација не успе. Овим се увек генерише исправан излазни код, без обзира на број истовремено „живих“ променљивих у програму.

3.7 Генерисање кода

Излазни .s фајл треба да има следећу структуру:

```
.globl main

.data
m1:      .word 6
m2:      .word 5

.text
main:
    la      $t0, m1
    lw      $t1, 0($t0)
    la      $t0, m2
    lw      $t2, 0($t0)
    add     $t0, $t1, $t2
```

слика 3.

3.8 Излаз програма

Поред генерисања излазне датотеке, програм на конзоли хронолошки исписује резултате сваке фазе превођења. Приказ обухвата:

- Успешност лексичке анализе
- Листу токена
- Резултат синтаксне анализе
- Граф тока управљања
- Све скупове анализе животног века променљивих
- Граф сметњи

- Резултат алокације ресурса, уз информацију о томе да ли је дошло до изливања (spilling)

4. Упутство за покретање

4.1 Покретање на Linux-у (Ubuntu)

Потребан је g++ компајлер. Његова инсталација је веома једноставна:

```
sudo apt update  
sudo apt install g++ build/essential
```

Пре него што компајлирамо и покренемо пројекат морамо се налазити у главном фолдеру пројекта који садржи сав код:

```
cd src
```

Компајлирање и покретање:

```
g++ *.cpp -o projekat  
./projekta
```

4.2 Покретање на Windows-у

Покретање на windows-у подразумева коришћење Visual Studio окружења.

Двокликом датотеке LexicalAnalysis.sln се отвара пројекат унутар окружења Visual Studio (обавезано је имати инсталиран *Desktop development with C++* пакет).

Изградња решења се врши комбинацијом тастера **Ctrl+Shift+B**, а покретање комбинацијом **Ctrl+F5**.

4.3 Избор улазне датотеке

Путања до улазне датотеке се подешава у main.cpp датотеци.

Линија кода: **std::string fileName = "../examples/multiply.mavn"**

Променом имена фајла бира се други тест пример (нпр. simple.mavn).

5. Коришћени тест примери

За проверу исправности рада преводиоца коришћени су различити тест примери, од којих сваки циља специфичан исход извршавања. Сви тестови се налазе у директоријуму **examples**.

5.1 Тест пример 1 – simple.mavn

Једноставан секвенцијални програм (без петљи) који учитава две меморијске променљиве из меморије и сабира их. Користи се за основну проверу исправности свих фаза преводиоца.

Резултат: регистар `t0` садржи вредност 11 ($6 + 5$). Свих 5 регистарских променљивих стаје у 4 расположива физичка регистра, без да дође до `spilling`.

5.2 Тест пример 2 – `multiply.mavn`

Сложенији програм који имплементира множење два броја поступком поновљеног сабирања, користећи петљу са условним скоком (`bltz`). Овај пример је кључан за проверу исправности графа тока управљања, анализе животног века у присуству петљи и механизма `spilling`-а.

Резултат: меморијска променљива `m3` садржи вредност 30 ($6 * 5$), детаљна анализа овог примера, укључујући и доказ је дато у наставку текста.

5.3 Тест пример 3 – `extended.mavn`

Програм који тестира све три додатне инструкције (`mul`, `and`, `beq`) у једном програму, заједно са произвољним именом функције (уместо `main`). Користи се за проверу да ли преводилац исправно генерише код за инструкције ван основног скупа од 10.

Резултат: свих 7 регистарских променљивих успешно добија физички регистар без иједног проблема (`spilling`-а).

5.4 Тест пример 4 – `lex_error.mavn`

Програм који намерно садржи карактер који не постоји у MABH језику (`@`). Користи се за проверу да ли преводилац исправно препознаје лексичку грешку и прекида анализу уместо да је игнорише.

Резултат: лексичка анализа исправно детектује грешку и враћа токен типа `T_ERROR` са вредношћу карактера, чиме се потврђује да преводилац не наставља даље са погрешним улазом.

5.5 Тест пример 5 – `syn_error.mavn`

Програм у коме недостаје обавезан зарез између операнада у инструкцији `la`. Користи се за проверу да ли синтаксни анализатор исправно прати дефинисану граматику и препознаје када редослед токена одступа од очекиваног.

Резултат: синтаксна анализа исправно препознаје неочекивани токен и пријављује грешку, уместо да погрешно парсира инструкцију или сруши програм.

5.6 Тест пример 6 – `undefined_var.mavn`

Програм који користи регистарску променљиву r2, која никада није декларисана. Користи се за проверу семантичке анализе.

Резултат: преводилац исправно препознаје да променљива није декларисана и пријављује грешку пре него што би покушао да генерише код за њу.

5.7 Тест пример 7 – unknown_instr.mavn

Програм користи инструкцију div која не постоји ни у основном скупу од 10 инструкција нити међу 3 додате инструкције. Користи се за проверу да преводилац не прихвата произвољне инструкције као да су валидне.

Резултат: синтаксна анализа одбацује непознату инструкцију као неочекивани токен и пријављује грешку.

6. Верификација

Поставља се питање да ли тест програм multiply.mavn уопште може бити исправно преведен коришћењем само 4 расположива регистра, с обзиром на то да садржи петљу у којој је истовремено потребно више променљивих.

6.1 Проблем

Почетна имплементација графа тока управљања повезивала је инструкције искључиво секвенцијално, без гране уназад за скок на лабелу петље (инструкције b, bltz i beq). Последица је била да анализа животног века није могла да препозна да променљиве r2, r4, r5 и r6 морају остати живе кроз целу петљу, пошто се њихова вредност користи у свакој наредној итерацији. Исправке графа тока управљања одрађена је тако да укључује грану повратка на лабелу петље (по узору на пример са Вежбе 9).

6.2 Доказ да је потребно 5 регистара

Након исправке, анализа животног века је показала да је тачно 5 регистарских променљивих живо истовремено између инструкција за сабирање и bltz скок.

Променљива	
r2	Множилац - потребан у свакој наредној итерацији (add r6, r6, r2)
r4	Граница петље - потребан у свакој наредној итерацији (sub r7, r5, r4)
r5	Бројач (addi r5, r5, 1)

r6	Резултат – чува се за следећу итерацију и упис у m3
r7	Потребан за инструкцију bltz

6.3 Решење

Проблем недовољног броја регистара је опште познат у теорији компајлера и решава се методом *spilling*-а, када граф сметњи није могуће обојити расположивим бројем боја, једна или више променљивих се уместо у регистру чувају у меморији, а у регистар се читавају само непосредно пре употребе.

У имплементираном решењу један регистар (t3) је резервисан искључиво као привремени регистар за *spill* операције, чиме алгоритму бојења графа остају на располагању 3 регистра (t0–t2) за променљиве које се могу трајно сместити у регистар. Променљива која не може добити ниједну од преостале три боје бива означена као просута (*spilled*), за њу се резервише меморијска локација у секцији *.data*, а генератор кода аутоматски пише инструкције *lw* пре сваке употребе и *sw* после сваке дефиниције те променљиве, користећи t3 као привремени регистар.

За програм *multiply.mavn*, алгоритам је одредио да променљиве r2 и r4 буду постављене у меморију као (*spill_r2*, *spill_r4*), док преосталих пет регистарских променљивих (r1, r3, r5, r6, r7) добијају сталне физичке регистре.

6.4 Провера у QTSpm симулатору

Исправност генерисаног кода и функционалност механизма изливања (*spilling*) успешно су верификовани симулацијом унутар симулатора QtSpim. Коначни извештај симулатора потврђује да су све вредности исправно израчунате и уписане у меморију, чиме је доказано да компајлер правилно руководи ограниченим регистарским ресурсима кроз *spill* меморијске променљиве.